Java Section 3

CLASSES, OBJECTS, INSTANCE VARIABLES AND METHODS

You will learn to define Java classes and work with objects

Classes, Objects, Instance Variables and Methods

1. **Object Oriented Terminology**
2. **Unified Modeling Language and the UML Class Diagram**
3. **Java Classes and Objects**
4. **Declaring a Java Class**
5. **Creating a Java Object**
6. **Attributes and declaring Java instance variables**
7. **Class methods and Static methods**
8. **Local variables**
9. **Constructors**

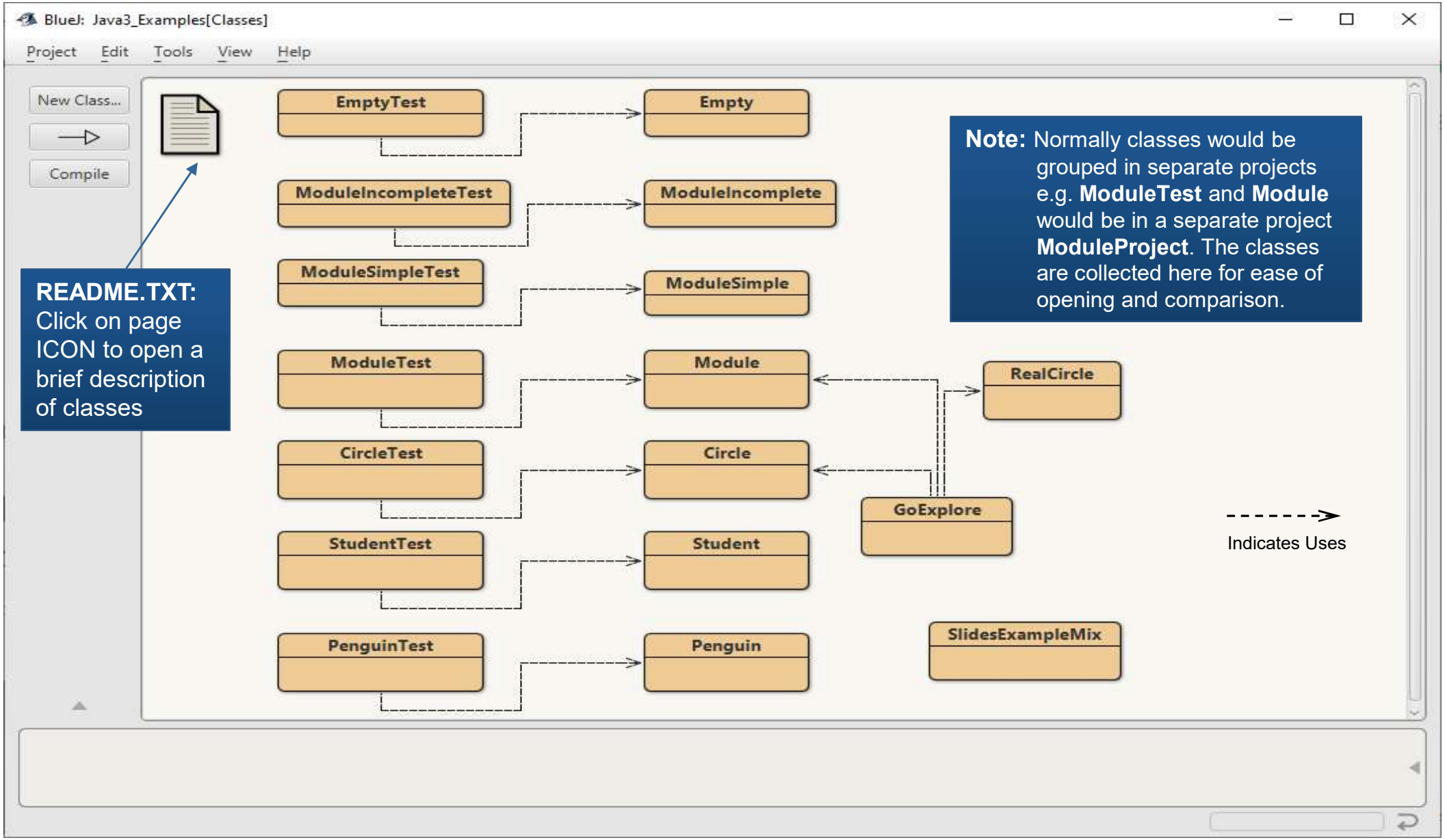
Lectures will emphasize the most important!

It is strongly recommended that you explore code by writing and executing the examples, read the various textbooks and complete the associated challenges.

JN

BlueJ: Java3_Examples[Classes]

Examples discussed in this section: Go Explore!



Introduction

- Classes in **Object Oriented (OO)** systems are mainly used to represent real world and abstract objects for computer processing
  - Allows partition of large complex systems for design/implementation
  - Individuals or teams can work on a single class or set of classes.
- Consider a university **timetable**, classes are necessary to represent:
  - **Person (Students, Lecturer, Tutor, …)**
  - Day, Time (Start, End), Week(s)
  - Module, Event
  - **Room**
  - The **Timetable** would be also be a class.

John
Wednesday 09:00 - 11:00
Wks: 2-12
EE4701 - LAB
B2-041

- Hence, OO programming is about:
  - Defining new classes and using existing classes
  - Creating objects of these classes
  - Testing these objects by calling their methods to check for correctness
  - Using these classes in an application

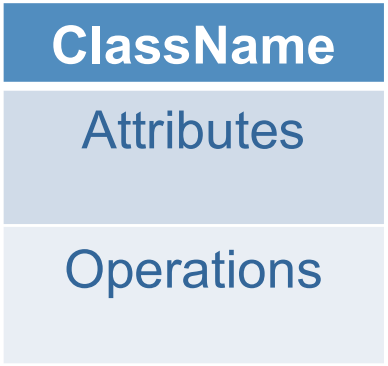
Classes, Objects, Instance Variables and Methods
Terminology

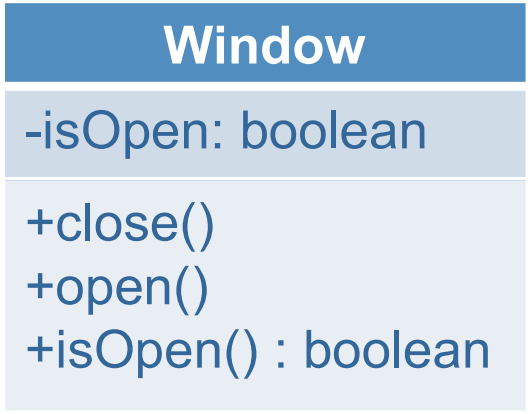
- A **class** is a generalised description for a collection of objects
  - *Specification* (analogy: the plans for a house).
- An **object** is an instance of a class (it has identity).
  - *Realisation* (analogy: the actual house(s) built from plans)
- A **class** can contains zero or more **instance variables**
  - An **attribute** maintains a feature or property of the object.
  - In Java classes **attributes** are stored in **instance variables**.
  - Each Java object (class instance) has its own set of **instance variables**.
  - The **instance variables** are manipulated by the methods.
- A **class** typically provides one or more **methods**
  - Methods correspond to the operations/tasks that the class supports
  - A method describes (and hides) the steps necessary to perform the task

UML (Unified Modeling Language)
Class Diagrams

- **UML** is the most widely used graphical notation for modelling Object Oriented (OO) systems.
  - UML is independent of any programming language.
- **UML Class Diagrams**
  - are used to describe classes and their relationships
  - most frequently used at the design stage
  - experienced programmes can use to program a class.
  - Attributes: **name : type**
  - Operations: **name(parameters) : return type**

    parameters: comma separated list of **name : type**
  - Prefix: **–** to indicate private, **+** public
- OO design tools exist to create UML diagrams.
  - Also, can **generate** parts or all of the code
  - For Java this would be the classes in the **.java** files.

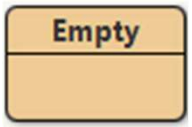




Java Classes
Overview

- Java has an extensive number of existing classes which you can use
  e.g. **Scanner**, **Math**, **Button**, **File**, **Clock** …
  - All available in the latest Java Application Programmers Interface (API)
  - The classes are grouped into **packages**
- Java is extensible: you can create new classes!
- Each class normally has an access modifier, and a unique identifier (name).
  - Access modifiers include **public** and **private** keywords.
  - Identifiers are meaningful names specified using **CamelCase**
  - Classes are normally **public** (usable by classes in other files)
  - For each class declaration that begins with the keyword **public**
    - The class must be stored in a file that has the same name as the class and ends with the **.java** file-name extension.
    - There must be only one **public** class in a file.

Java Classes
Declaring a class



Empty.java

- Class declarations include:

accessModifier class identifier { }

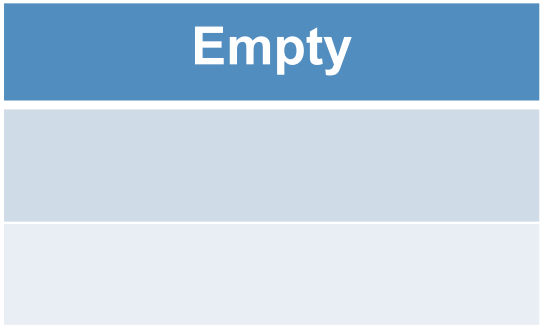
accessModifier	**public** or **private**
class	keyword
identifier	a user defined name
{ }	left & right braces marking the begin and end

- Let's create an empty class:

Java

```
public class Empty
{

}
```

UML



Creating a Java Object
Instantiation

- We **create** objects from classes, known as **instantiating objects**
  - Terminology: **instantiate** an **object** from a **class**
  - Analogy: create (*instantiate*) a house (*an object*) from a plan (*a class*)
- Use the keyword **new** followed by the name of the class and parentheses, *which can contain optional arguments or be empty:*

ClassName objectReference = **new** ClassName(optionalArguments);

- If the ClassName file isn't in the same folder, we must first **import** it, by prefixing the package name **import packageName.className**;

```
import java.util.Scanner;
public class FindMax
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);
```

Recall the **FindMax** example created an ***instance*** of a **Scanner** which was named **keyboard**.

Creating a Java Object
Calling an Object Method

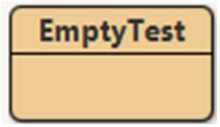
- Classes provide operations which are implemented as methods.
- We call object methods by
  1. selecting the **object** using its **objectReference**
  2. selecting the **method** using a dot **.** followed by the **methodName** with optional arguments.

objectReference.methodName(optionalArguments);

```
public class FindMax
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);

        System.out.print("Enter 1st number: ");
        int number1 = keyboard.nextInt();
```

In the **FindMax** example a Scanner object is created and the Scanner method **nextInt()** is called to get an integer .

Instantiating an Object
of class Empty



EmptyTest.java

- Most often, we use a separate class to test a class
- **EmptyTest** will test class **Empty**

```
public class Empty
{

}
```

using the main method.

```
public class EmptyTest
{
    public static void main(String[] args)
    {
        Empty emptyObject1 = new Empty();    // Create an Empty object
        Empty emptyObject2 = new Empty();    // Create another one
        // two objects created but there are no methods in the class to call
    }
}
```

Attributes and instance variables

- A **class** can contain zero or more **attributes**
- An **attribute** maintains a feature or property of the object
  - Implemented in java as an **instance variable**.
  - Each created **object** will have its own set of **instance variables**.
- Instance variable declarations specify access, a type and identifier:

accessModifier type identifier;

accessModifier	**private** (this class only) or **public** (all classes)
type	a primitive or reference type
identifier	the user defined name

- Normally access is **private** to ensure **data/information hiding**.
- The type indicates what will be stored (e.g. **int** for an integer)
- The Java naming convention for the identifier is : **camelCase**

Examples:

```
private String name;
private int id;
```

camelCase is the practice of writing phrases without spaces or punctuation, separating words with capitalised letters, where the first word starts with lower case.

Declaring an instance variable

Incomplete example for a Module title

ModuleIncomplete.java

- Consider we want to create a **class** to represent a university module
  - For now, let's store a **title** for each module e.g. "Mad Software 1"
  - Each object instantiated will have its own **title**

```
public class ModuleIncomplete
{
    private String title;    // store the module title
}                            //  e.g. "Mad Software 1"
```

ModuleIncomplete
-title : String

- Let's create an object. We cannot do much, as instance variable is private!

```
public class ModuleIncompleteTest
{
    public static void main(String[] args)
    {
       ModuleIncomplete module1 = new ModuleIncomplete();    // Create an object
       //System.out.println( module1.title ); // syntax error as title is private
    }
}
```

Java Variable Types
Recall Primitive Types

- Java has eight **primitive types** portable across all platforms
  byte, short, int, long, float, double, char, boolean
- Java is a **strongly typed language** .
  - Java requires all variables to specify their **type** when first declared.
  - Each Java variable can be declared as a **primitive type** or a **reference type**
- **Initialization:** (the value a variable has when first created)
  - **Local variables** (created within methods) are never initialized
  - **Instance variables** are initialized to their default initial value by type:
    0 for **byte, short, int, long, float, double, char**
    false for **boolean**

Part of Java's *WORA*
Write Once, Run Anywhere.

Examples of Primitive type variables:

1. Local variables	**int number1;**	in the **main** method of class **FindMax**
	int maximum;	
2. Instance variable	**private int id;**	e.g. declared in a class **Student**

Java Variable Types
Reference Types

- All classes are **reference types** (*aka* non-primitive types)
  - Objects are accessed through **variables** of **reference types** (aka **references**)
  - **Reference-type** variables are used to invoke the methods of an object.
  - **Initialization:**
    - **Reference-type** variables are always initialized to the value **null**.

Examples of reference type variables:

1. Local variables	Scanner **keyboard =** new Scanner(System.in);	in FindMax
	String **greeting** = new String("Hello");	in StringDemo
2. Instance variable	private String **title;**	in class Module

Class methods
Overview

- Class methods provide the operations of the class.
- Class instance variables are accessed through class methods.
- Class methods may be **void** or return a **primitive** or **reference** type:

```
public void methodName( optionalParameters )        // void if nothing returned
{
   statements
}
public returnType methodName( optionalParameters )
{
    statements
    return expression;
}
```

- Various standardised methods are created in classes

get method(s)	to return the value of an instance variable
set method(s)	to change the value of an instance variable
toString method	to return a String representation with object details

- Methods can be in any order and have usually **public** access.

Class methods
Defining a method

- A method is defined by providing a method declaration and method body.
- A class method definition has **five parts** (for now):
  1. An **access modifier** —such as **public**, **private**
  2. The **return type** - the data type of the value returned by the method, or void if the method does not return a value.
  3. The **method name** - should follow the Java ***camelCase*** convention for naming.
  4. An optional **parameter list** in required parenthesis

     A comma-delimited list of input parameters, preceded by their data types, enclosed by parentheses, **( )**. If there are no parameters, use empty **( )**.
  5. The **method body**, enclosed between braces, { }

     Contains the method's statements, including the declaration of local variables.
- It is normal to document a method using a **javadoc** module header comment /** … */

Method Comment:	**/** javadoc comment to describe method*/**
Method Declaration:	**accessModifier returnType methodName(optionalParameterList)**
Method Statements:	**methodBody**

Class methods
Parameters and Arguments

- **Parameters** are specified in a method's **parameter list**:

```
public void display( int a, int b, int c ) { … }   // e.g. print as (a,b,c)
```

  - The **parameter list** is part of the method header
  - Use a comma-separated list if more than one parameter
  - Each parameter must specify a type and a name

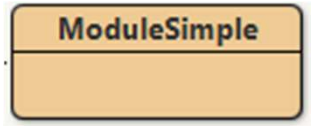
- The number of **arguments** in a method call *must* match the number of parameters in the method declaration

```
display( 2+5, 4*(3-1), 10/2);     // assumes display is in same class

ClassName.display(7, 8, 5);   // assumes display in class ClassName
```

  - The types of the arguments in a method call *must be* consistent with the types of the corresponding parameters in the method declaration.
  - Valid expressions can be used for each argument.

Class methods

Creating get and set methods



ModuleSimple.java

- A **get** and **set** method is defined for each instance variable (fields)

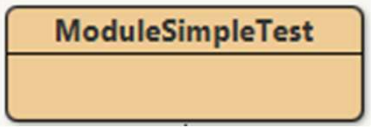
  e.g. **getTitle** and **setTitle** for the **title** instance variable

ModuleSimple
-title : String
+getTitle( ) : String +setTitle(newTitle:String)

```
public class ModuleSimple
{
    private String title;       // instance variable for the module title
    // get/return the title of the invoking object
    public String getTitle()
    {
        return title;
    }
    // set the title of the invoking object to the parameter newTitle
    public void setTitle(String newTitle)
    {
        title = newTitle; // Assign the newTitle to instance variable title
    }
}
```

Class methods

Executing get and set methods



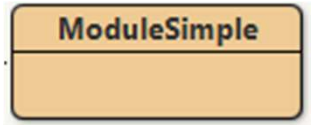
ModuleSimpleTest.java

```
import java.util.Scanner;
public class ModuleSimpleTest
{
    public static void main(String[] args)
    {
        ModuleSimple module = new ModuleSimple();

        System.out.println( module.getTitle() );
        module.setTitle("Mad Software 1");
        System.out.println( module.getTitle() );

        Scanner keyboard = new Scanner(System.in);
        System.out.print("Enter a module title: ");
        module.setTitle ( keyboard.nextLine() );
        System.out.println( module.getTitle() );
    }
}
```

```
null
Mad Software 1
Enter a module title: Hatter Software 2
Hatter Software 2
```

Class methods

get and set method patterns



ModuleSimple.java

Similar for each instance variable

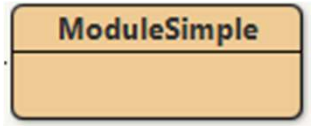
```
private type name;

public type getName()
{
    return name;
}

public void setName(type pName)
{
   // Optional checks go here to
   // ensure the value of pName is ok
    name = pName;
}
```

Prefix **p** for parameter, _ or **new** used too

```
public class ModuleSimple
{
    private String title;

    public String getTitle()
    {
        return title;
    }

    public void setTitle(String pTitle)
    {
        title = pTitle;
    }
}
```

Class methods

Creating the toString method



ModuleSimple.java

- The toString method: **String toString()** returns a String representation of the invoking object in the form:

className{**iv1**=value_iv1, **iv2**=value_iv2, …}

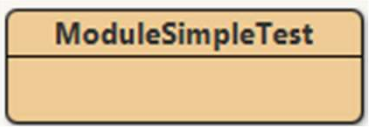
where **iv1** *is name of 1st instance variable, and* **value_iv1** *its value etc*

```
public class ModuleSimple
{
    private String title;

    public String toString()
    {
        return "ModuleSimple{" + "title=" + title + '}';
    }
}
```

ModuleSimple
-title : String
+getTitle( ) : String +setTitle(newTitle:String) +toString)( ) : String

Class methods

Executing the toString method



ModuleSimpleTest.java

```
import java.util.Scanner;
public class ModuleSimpleTest
{
    public static void main(String[] args)
    {
        ModuleSimple module = new ModuleSimple();

        System.out.println( module.getTitle() );
        module.setTitle("Mad Software 1");
        System.out.println( module.getTitle() );

        Scanner keyboard = new Scanner(System.in);
        System.out.print("Enter a module title: ");
        module.setTitle ( keyboard.nextLine() );
        System.out.println( module.getTitle() );

        System.out.println( module.toString() );
        System.out.println( module ); // object name invokes toString
    }
}
```

- Invoke the object **toString** method

  public String toString()

  using:

  objectReference.toString()

- Or implicitly call **toString** by using the **objectReference** alone where a String is expected.

```
null
Mad Software 1
Enter a module title: Madder S/W 2
Madder S/W 2
ModuleSimple{title=Madder S/W 2}
ModuleSimple{title=Madder S/W 2}
```

Instantiating an Object and Calling a Class Method
Summary

- When an object is created, a unique reference to that object is created and normally stored in a variable of type ClassName

```
ClassName objectReference = new ClassName( );
```

- We call/invoke a method to perform its task

```
objectReference.methodName( );
objectReference.methodName( argumentList );
```

- Examples: create module object and invoke methods

```
ModuleSimple module = new ModuleSimple();
module.setTitle("Mad Software 1");

System.out.println( module.toString() );
```

Static methods
Overview

- Static methods can be called without creating an object of the class.
- Used where the method **does not** access any instance variables.
- Include **static** keyword after the access keyword.
- Static methods may be **void** or return a **primitive** or **reference** type:

```
public static void methodName( optionalParameters )
{
   statements
}

public static returnType methodName( optionalParameters )
{
    statements
    return expression;
}
```

Local Variables

SlidesExampleMix

- **Local variables are variables declared in the body of method**
  - Must declare before use and specify its type and a name.
  - Local variables are not initialised: the value is undefined until assigned.

```
public class SimpleMethod
{
    public static int square(int x)
    {
        int y;
        y = x * x;
        return y;
    }

    public static void main(String[] args)
    {
        int number = 10;
        System.out.print("Square of " + number );
        System.out.println(" is " + square(number) );
    }
}
```

Parameter **x** of type **int**

Local variable **y** of type **int**

Local variable **number** of type **int**

Argument **number** for parameter **x**

```
Square of 10 is 100
```

Inputting strings and primitive types
Scanner methods

- **Scanner** methods to read a line or a word

  nextLine()
  - Reads characters typed by the user until the newline character is encountered
  - Returns a **String** containing the characters up to, but not including, the newline
  - Pressing *Enter* inserts a newline character at the end of the characters and submits the string, but the newline is discarded by **nextLine**.

  next()
  - Reads individual words
  - Reads characters until a white-space character is encountered, then returns a **String** (the white-space character is discarded).

- **Scanner** methods for primitive types

  nextInt(), nextByte(), **nextLong(), nextShort()**

  nextDouble(), **nextFloat()**,

  nextBoolean() accepts only **true** or **false** (case insensitive)

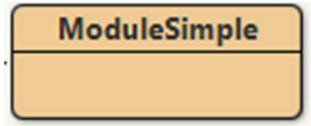
Constructors

- Constructors are used to initialise one or more object instance variables
  - when an object is created, with the **new** keyword
- Java requires one or more constructors for every class
  - Java will provide a **default no-argument constructor** if none is provided
  - The default constructor will initialize instance variables to default values
- A **full argument** constructor is one that has a parameter for each instance variable
- The constructor method name is the name of the class e.g. for **Module**

```
public Module( )      {…}                          // No parameters
public Module (String title) {…}                   // String parameter
public Module (String code, String title) {…}      // Two String parameters
```

- Note there is no return type specified for constructors.

No argument constructor
class ModuleSimple



ModuleSimple.java

- Recall we didn't create a constructor earlier for ModuleSimple class but still could create objects of the class
  - Java created the **default no-argument constructor**
- Below we explicitly add a **no-argument constructor**

```
public class ModuleSimple
{
    private String title;    // instance variable for the module title

    // No argument constructor.  If not explicitly specified
    // Java creates an implicit default no-argument constructor
    public ModuleSimple()
    {
        title = null;
    }
}
```

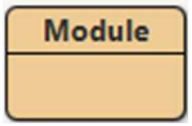
ModuleSimple
-title : String
<<constructor>> ModuleSimple()

Complete Class
class Module

- Introducing class Module *see Module.java & ModuleTest.java*
  - Two String attributes (Java instance variables)
    code for the *module code* and **title** for the *module title*
  - Operations (Java methods) include get/set for **code** and **title**, toString
  - Full argument constructor

Module
- code : String - title : String
+ setCode(pCode: String) + getCode( ) : String + setTitle(title : String) + getTitle( ) : String + toString( ) : String + displayMessage( ) : String <<constructor>> Module(pCode: String, pTitle: String)

Complete Class
class Module with Full Argument Constructor

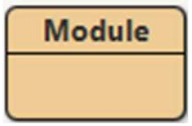


Module.java

Module
-code: String -title : String
<<constructor>> Module(pCode:String, pTitle:String)

```
public class Module
{
    // Instance variables to
    private String code;
    private String title;

    /**
     * The full argument constructor (one parameter for each instance variable)
     * @param pCode the module code
     * @param pTitle the module title
     */
    public Module(String pCode, String pTitle)
    {
        code = pCode;      // Assign parameter pCode to instance variable code
        title = pTitle;    // Assign parameter pTitle to instance variable title
    }
```

this keyword



Module.java

- Within a non-static/instance method or in a constructor, **this** refers to the current object, whose method or constructor is being called.
- **this** can be used to refer to any member of the current object and is normally used to refer to the instance variables.

```
/**
 * The full argument constructor (one parameter for each instance variable)
 * @param code the module code
 * @param title the module title
 */
public Module(String code, String title)
{
    this.code = code;      // Assign parameter code to instance variable code
    this.title = title;    // Assign parameter title to instance variable title
}
```

A frequent mistake is to type:

title = title;

```
public Module(String pCode, String pTitle)
{
    code = pCode;      // Assign parameter pCode to instance variable code
    title = pTitle;    // Assign parameter pTitle to instance variable title
}
```

which just assigns the parameter to itself.
The value of the instance variable is not changed.
Hence, programmers often use different names for instance variables and parameters.

Review of basic concepts

- **Types**
  - Primitives Type *versus* Reference Type
- **Variables**
  - Each variable must have a **type:** either a **primitive type** or a **reference type**
  - **Local variables** are created within methods (not initialised).
  - **Instance variables** are declared in classes to represent attributes.
  - Each variable has a **Scope** referring to the region of code where it can be accessed.
- **Methods**
  - Instance method *versus* Static method
  - Getter method, Setter method, toString method
  - Overloaded methods can have the same name but different parameter lists.
- **Constructors**
  - Used only to create and initialize objects.
  - No argument (parameter) constructor, Full argument, n-argument constructors
  - Default no-argument constructor

A Java application with Multiple Classes

- Java Application
  - One class has a main method where execution begins

    public static void main(String[] args)

  - The main method accepts an optional array of String arguments.
- Simple applications can be completed in a single class
- Most applications require multiple classes
- You can create multiple classes in a single file
  - But there can only be one **public** class in a **.java** file
- More normal is to have multiple files
  - With a single class in each file of type **public**

Notes on Import Declarations

import vs fully-qualified class names

- Classes are normally stored in a **package**, and to use a class we must specify where to find it
  - Either **import** the class or use the fully qualified class name
- `java.lang` package is implicitly imported into every program
  - Classes **System** and **String** are in the package **java.lang**
  - The compiler effectively adds an **import java.lang.*;** to each source file
- **Default package**
  - Contains classes compiled in the same directory
  - Implicitly imported into source code of other Java files in the same directory
- **Fully-qualified class names**
  - Contain the package name and the class name
  - Imports are unnecessary if fully-qualified names are used

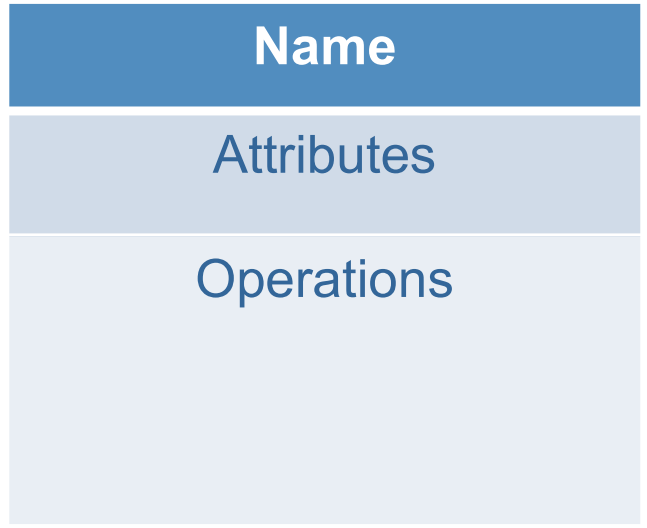
```
java.util.Scanner keyboard = new java.util.Scanner( System.in );
```

- **import** is used by most programmers e.g. **import java.util.Scanner;**

```
Scanner keyboard = new Scanner( System.in );
```

Completing the UML Class Diagram

1. Name the **class**, *in the first row/compartment*
2. Declare the **attributes**
3. Declare the **get/set** operations *per attribute*
4. Declare a **toString** operation
5. Declare the **constructor(s)** «*constructor*»



UML class structure

- Adopt Java naming conventions
  - Start uppercase for Class name, lowercase for attributes/operations
- The **attributes** will be defined as fields in the corresponding Java class
- The **operations** will be defined as methods in the corresponding Java class
- Attributes are normally **private** (*prefix* **-**), operations **public** (*prefix* **+**)
- The **toString** method returns a String representation of an object's details: class name, attribute names and values e.g. "Student:{name=Ada, id=42}"

UML Example 1:
Student

Complete a UML class diagram for a class to represent a student.

- Assume that the attributes are: **id**, **name**.
- Include the **set/get** and **toString** methods, and a full argument constructor

1. Name the class
2. Declare the **attributes**
3. Declare the **get/set operations**
4. Declare a **toString operation**
5. Declare full argument **constructor**

Name
Attributes
Operations

UML class structure

UML Example 1: class Student
Attributes and get/set operations

Complete a UML class diagram for a class to represent a student.

- The attributes are: **id** and **name**
- Declare the **set** and **get** operations
- Declare the **toString** operation
- Declare a full argument **constructor**

Solution

1. Name the class, *in the first row*
   - Adopt Java naming conventions
2. Declare the **attributes**, *in the second row*
   i. Write a list of the attribute names
   ii. Prefix: **– for private** or **+** for **public**
   iii. Select a type for each (**int** or **String**)
3. Declare the set/get **operations**, *in the third row*
   i. First write in the operation names
   ii. Prefix: **+ for public** or – for **private**
   iii. Add the parameter for the **set** and return type for **get**

*Note it is common to use the same instance variable name for the parameter or to prefix with **p**, **new**, _*

Student
- id : int - name : String
+ setId(id : int) + getId( ) : int + setName(name: String) + getName( ) : String

UML Example 1: class Student
toString and constructors

Complete a UML class diagram for a class to represent a student.

- The attributes are: **id** and **name**
- Declare the **set** and **get** operations
- Declare the **toString** operation
- Declare a full argument **constructor**

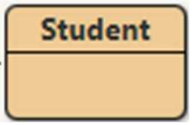
Student
- id : int - name : String
+ setId(id : int) + getId( ) : int + setName(name: String) + getName( ) : String + toString( ) : String <<*constructor*>> Student(id : int, name : String)

4. Declare a toString operation
   - returns a String representation of an object's class attributes

   e.g. "Student: name=Ada, id = 42"

5. Declare a full argument constructor
   - Always same name as class
   - Two parameters for id and name

Java class Student code extracts:
Class Declaration and Instance Variables



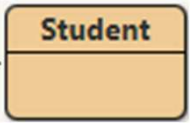
Student.java

Student
- id : int - name : String

```
/**
 * The class Student represents a university student
 * @author John.Nelson
 * @version 3 Oct 2017
 */
public class Student
{
    // Instance variables to store information for a student
    private int id;              // e.g. 17123456
    private String name;         // e.g. "Fred Flintstone"
```

Note: ***Every created student object (i.e. instance of a class) will have its own copy of the class instance variables***

Java class Student code extracts:
Constructor



Student.java

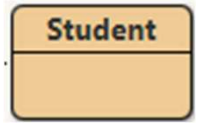
```
public class Student
{
    // Instance variables to store info
    private int id;              // e.g.
    private String name;         // e.g.

    /**
     * The full argument constructor
     * (one parameter for each instance variable)
     * @param pId   the student id
     * @param pName the parameter presented for the student name
     */
    public Student(int pId, String pName)
    {
        id = pId;       // Assign parameter pId to instance variable id
        name = pName;
    }
```

Student
- id : int - name : String
<<constructor>> Student(id : int, name : String)

The constructor is called when an object is created:

Student stu1 = new Student(1943, "Jo");

Java class Student code extracts:
get and set methods for name



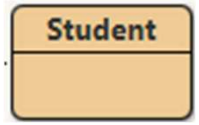
Student.java

Student
- name : String
+ setName(name: String) **+ getName( ) : String**

```
private String name;          // e.g. "Fred Flintstone"

/**
 * Get the name of a student
 * @return the name of the student object which invoked this method
 */
public String getName()
{
    return name;
}

/**
 * Set the name of the student object which invoked this method
 * @param pName the student's name to assign to the instance variable name
 */
public void setName(String pName)
{
    name = pName;    // Assign parameter pName to instance variable name
}
```

Note use of **p** prefix to distinguish parameter name

Java class Student code extracts:
get and set methods for id

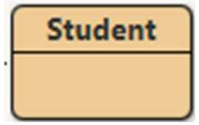


Student.java

```
private int id;              // Instance variable to store id number
/**
 * Get the id of a student
 * @return the id of the student object which invoked this method
 */
public int getId()
{
    return id;
}

/**
 * Set the id of the student
 * @param id the student's id to assign to the instance variable id
 */
public void setId(int id)
{
    // Note parameter and instance variable identifiers are both id
    this.id = id; // Use 'this' to differentiate the instance variable
}
```

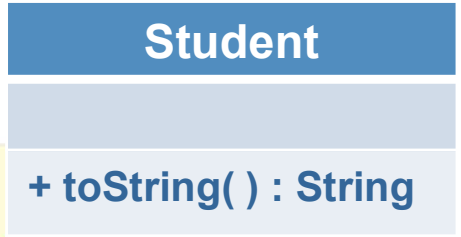
Student
- id : int
+ setId(id: int) + getId( ) : int

Java class Student code extracts:
toString method



Student.java

// **toString** builds and returns a String with the class name
// and a list of the instance variable names and their values




```
/**
 * Returns a concise readable string representation of the object
 * @return  a string representation of the object.
 */
public String toString()
{
    return "Student{" + "id=" + id + ", name=" + name + '}';
}
```

// When invoked on an object, the **toString** returns a String which then can be printed.

```
Student stu1 = new Student( 180000001, "Fred" );
System.out.println( stu1.toString() );
```

// stu1.toString() will return String "Student{ id=180000001, name=Fred}“

// Use println to display:

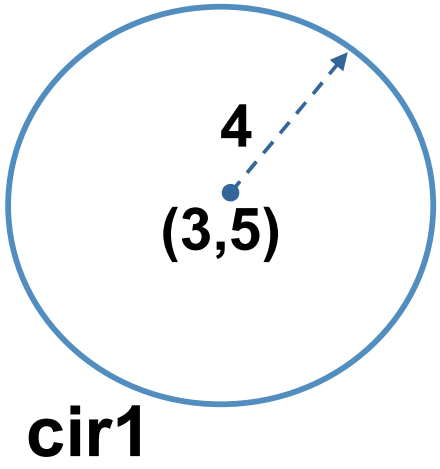
```
Student{id=18000001, name=Fred}
```

UML Example 2:
Circle

Complete a UML class diagram for a class to represent a circle.

- Assume that the attributes are: **name, x, y, radius**.
- Include the **set/get** and **toString** method, and two constructors

1. Name the class
2. Declare the attributes
3. Declare the get/set operations
4. Declare a toString operation
5. Declare two constructors



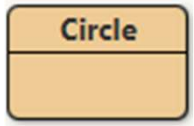
Name
Attributes
Operations

UML Example 2: class Circle
UML Class Diagram

Given the attributes: **name, x, y, radius**, the Class diagram can be completed.

	Circle
Class Name	
Attributes	- **name** : String - **x** : int - **y** : int - **radius** : int
Operations	
set and get	+ **setName**(name: String) + **getName**( ) : String + **setX**(x : int) + **getX**( ) : int + **setY**(y : int) + **getY**( ) : int + **setRadius**(r : int) + **getRadius**( ) : int
toString	+ **toString**( ) : String
No argument constructor	<<*constructor*>> **Circle**( )
Full argument constructor	<<*constructor*>> **Circle**(name : String, x : int, y : int, r : int)

Set method checking
Introducing IllegalArgumentException



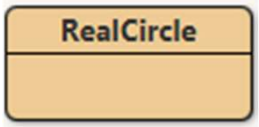
Circle.java

- Normally **set** methods perform checks to ensure that the instance variables are not assigned illegal values e.g. a circle with radius <= 0.
- Starting out, we may print a message informing the user.
  - Such messages have very limited usefulness.
- More common is to raise an exception (*more on exceptions later)*
  - *For now, throwing an* **IllegalArgumentException** will suffice.

```
/**
 * Set the radius
 * @param r the radius
 * @throws IllegalArgumentException if the r parameter is invalid
 */
public void setRadius(int r)
{
    if (r > 0)
        radius = r;
    else
        throw new IllegalArgumentException("radius must be greater than 0");
}
```

Circle with real number attributes

double type for x, y and radius



RealCircle.java

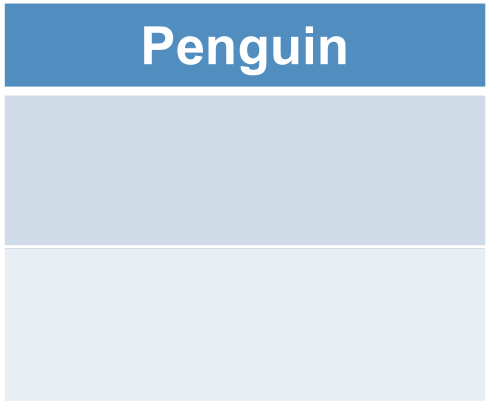
```
public class RealCircle
{
    private String name;
    private double x;
    private double y;
    private double radius;

    // No parameter (aka no argument) constructor
    public RealCircle()
    {
        name = null;
        x = 0;
        y = 0;
        radius = 0;
    }

    // Full (four) parameter constructor
    public RealCircle( String pName, double pX, double pY, double r)
    {
        name = pName;
        x = pX;
        y = pY;
        radius = r;
    }
```

The class **RealCircle** is similar to class **Circle** except we use real numbers instead of integers for x, y and radius.

UML Example 3:
Penguin

Complete a UML class diagram for a class to represent a penguin.

- Assume that the attributes are: name, place, height, xPos, yPos.
- Include the **set/get** and **toString** operations, and a **constructor**



1. Name the class
2. Declare the **attributes**, and identify their types
3. Declare the **set/get operations**
4. Declare the **toString** method
5. Declare a suitable **constructor** (*you decide the parameters*)

   <<*constructor*>> Penguin(????)
6. Declare a method **setLocation** which will set both **xPos** and **yPos**
7. Declare a method **getHeightDescription** to return a String description

   "Tall" >= 80, 50<= **"Average"** <80, 0 < **"Small"** <50, else **"Height is not valid"**

UML Example 3: class Penguin
UML Class Diagram

Given the attributes:

name, place, height, xPos, yPos

most of the class diagram can be completed: class name, attributes, set/get operations, toString and constructor.

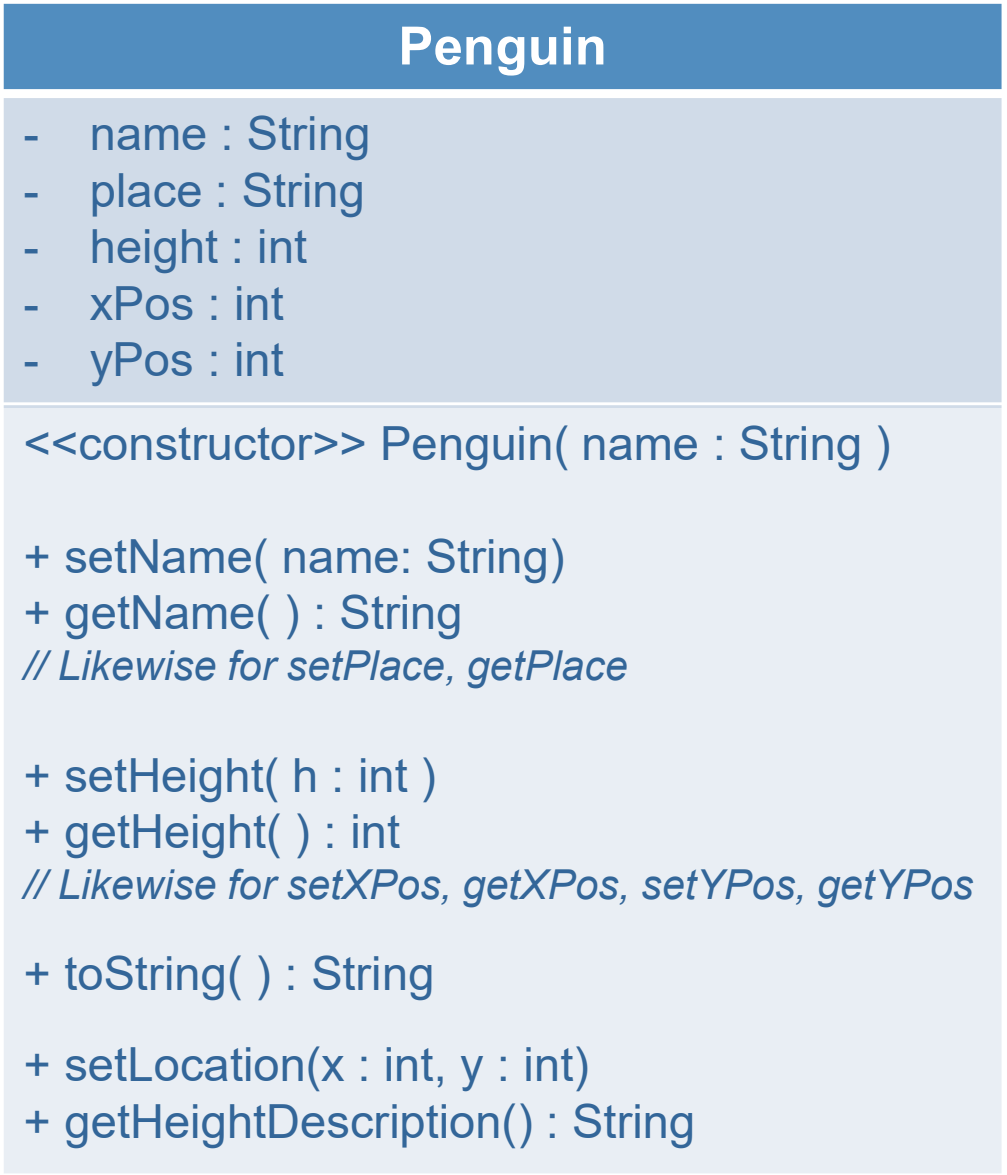
Then declare the extra operations:

setLocation

getHeightDescription

With the earlier description of these two methods there is now enough information to code the class in Java.

We can test it by using a test class **PenguinTest** to construct some Penguin objects and invoke the methods.



Go Explore

class in Java3_Examples

GoExplore.java

- Many of the introduced concepts are captured in the **GoExplore** class
  - Create strings explicitly and implicitly
  - Call methods and demonstrate argument passing.
  - Create objects and call methods of **Circle**, **RealCircle** and **Module** classes
  - Invoke various class methods
  - Create static method **public static int min(int a, int b, int c, int d)**

- Read the code once and execute the main method. Observe the output.

- Reread the code a few times and identify any code requiring clarification. *Note any and address them at lecture/labs.*

- **Modify the code:**
  1. Create other objects and set their attributes (instance variables). Display them.
  2. Extend the **min** method to take 5 parameters and test it. Note the pattern.

Homework: Design Exercise (Recall?)
email Object and Class

Consider a recent email that you have sent or received.

Think of it as an object with attributes and operations.

- What were the attributes associated with it?
  - List at least 6 attributes
- What operations could you do with it?
  - List at least 6 operations
- Define a UML class diagram that will represent a general email.

Homework: Design Exercise
email Attributes

- What attributes?

	Instance Example
From	*Fred*
To	*Willllll-ma*
Subject	*The dog!*
Date&Time of Send and/or Receive	*09 Oct 1960 13:00*
Body of email	*Are you taking Dino to Bedrock?*
Size	*200 (Kg!)*
Attachments….	*A few Stones*
Read acknowledgement requested	*True*
Mail was read	*False*

- Consider what Java types you use to implement?

Homework Email Design Exercise
Object and Class: Operations and Class

- What operations?

Send
Reply
Forward
Open
Print

Outline a UML diagram to represent the class

1. ***Name the class***
2. ***Define 6 attributes***
3. ***List 6 operation names***

Move (to Folder)
Mark as Read
Mark as Unread
Delete
Edit
…